AWS VPC Peering Project

Connecting Two VPCs and Verifying Private Communication

Name: PRIYANSHU TIWARI

Internship: AWS Internship Batch

Date: 24/10/2026

1. Objective

To create two separate Virtual Private Clouds (VPCs) in AWS, connect them using a VPC Peering connection, and verify that EC2 instances in the two VPCs can communicate with each other using private IP addresses.

2. Services Used

Amazon VPC, Subnets, Internet Gateway, Route Tables, Security Groups, VPC Peering, Amazon EC2.

3. Architecture

Component	VPC-A	VPC-B
VPC CIDR	10.0.0.0/16	10.1.0.0/16
Public subnet	10.0.1.0/24	10.1.1.0/24
Private subnet	10.0.2.0/24	10.1.2.0/24
Internet Gateway	igw-A	igw-B
EC2 (public)	A-Public-EC2	B-Public-EC2
EC2 (private)	A-Private-EC2	B-Private-EC2

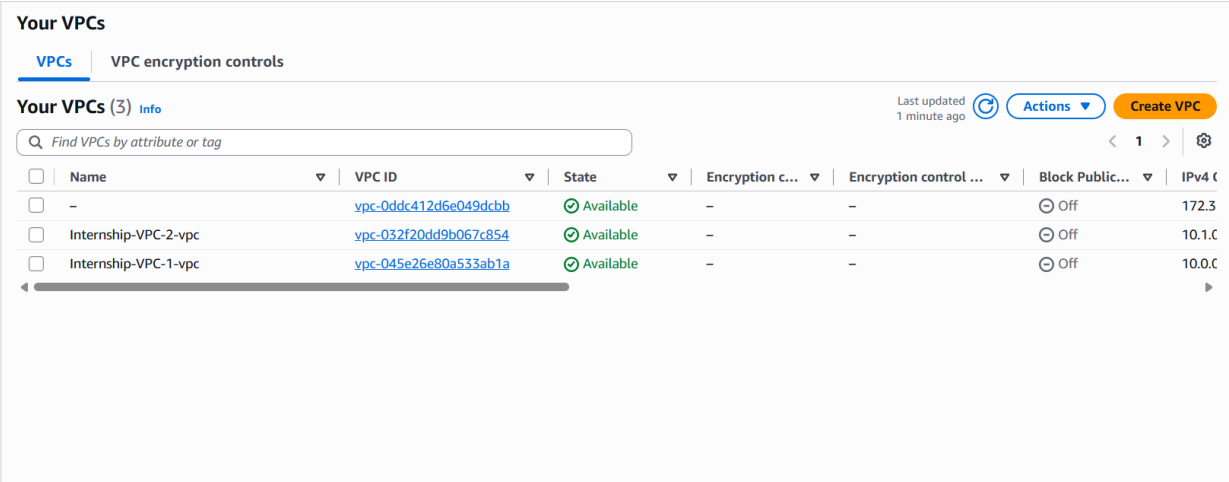
Note: the two VPC CIDR ranges must not overlap, otherwise peering cannot be created.

Step 1: Create the two VPCs

VPC console, Create VPC, choose 'VPC only'. Create **VPC-A (10.0.0.0/16)** and **VPC-B (10.1.0.0/16)**.

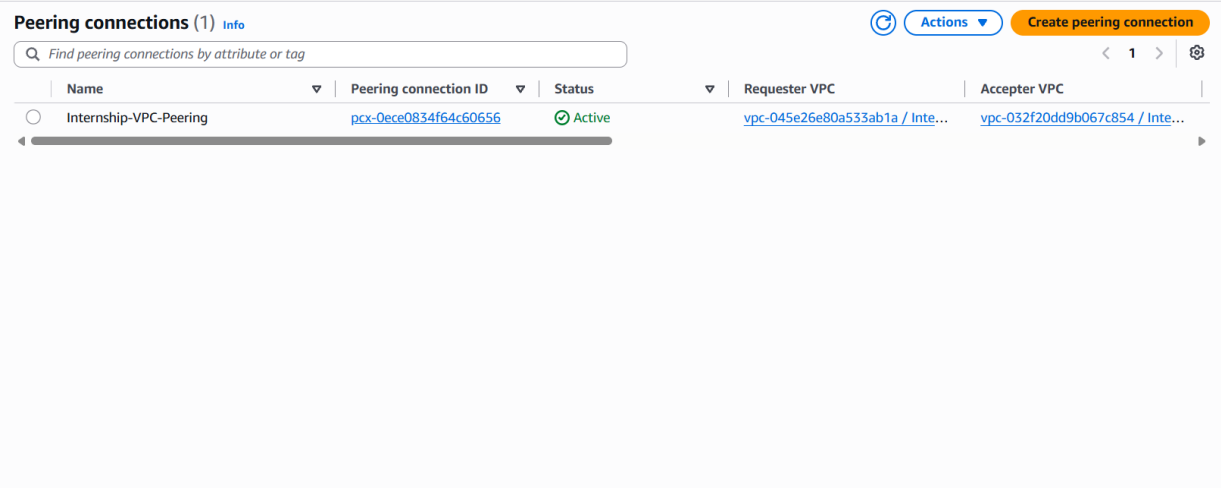
Optional AWS CLI equivalent:

```
aws ec2 create-vpc --cidr-block 10.0.0.0/16 --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Val
aws ec2 create-vpc --cidr-block 10.1.0.0/16 --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Val
```



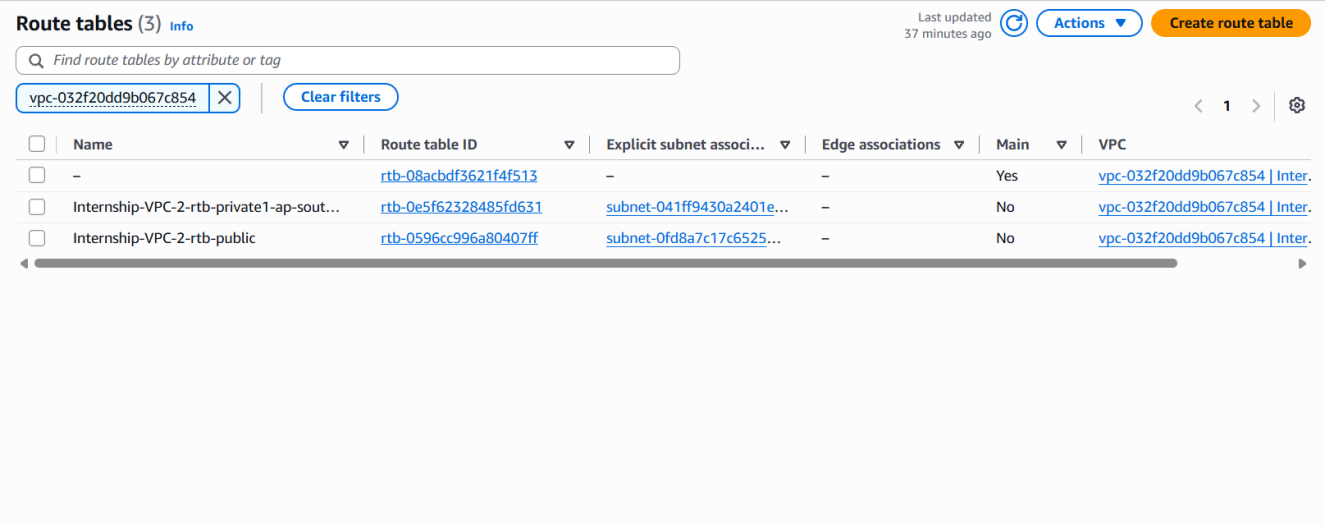
Step 2: Create subnets (1 public + 1 private in each VPC)

VPC-A: public 10.0.1.0/24, private 10.0.2.0/24. VPC-B: public 10.1.1.0/24, private 10.1.2.0/24.



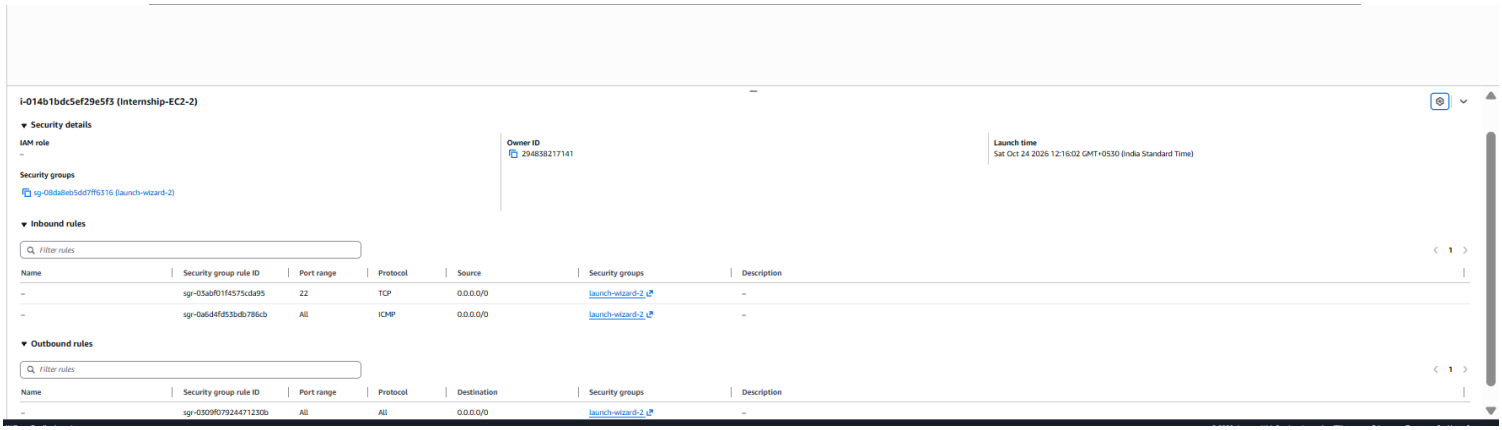
Step 3: Internet Gateway and public route table

Create an Internet Gateway for each VPC and attach it. In the public subnet route table add **0.0.0.0/0** pointing to the Internet Gateway, and associate it with the public subnet. Enable 'Auto-assign public IPv4' on the public subnets.



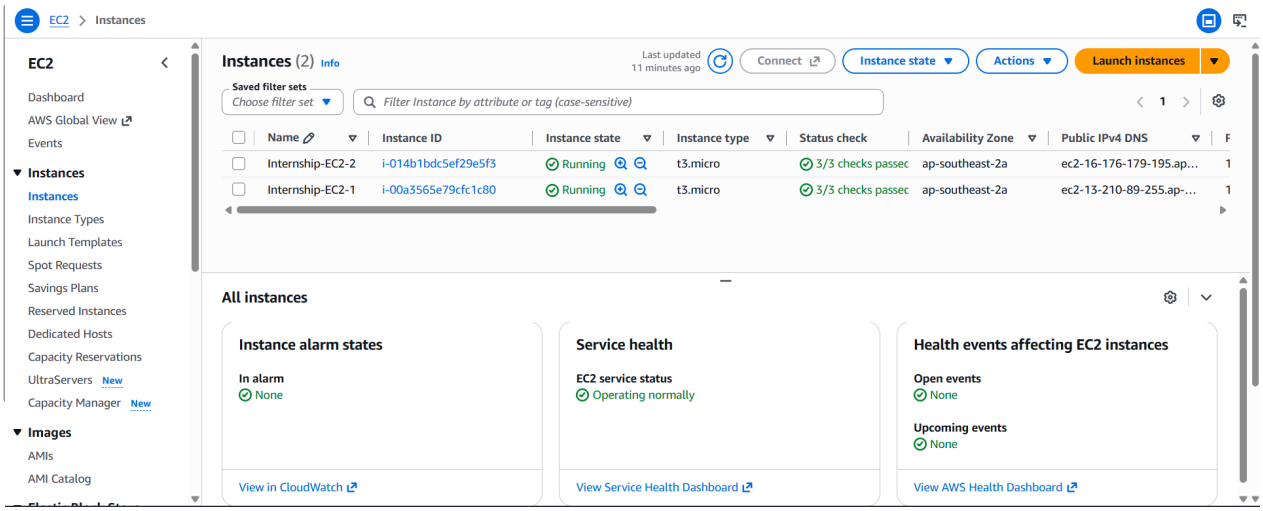
Step 4: Security Groups

Allow **SSH (22)** from your IP, and **All ICMP - IPv4** from the other VPC's CIDR (10.1.0.0/16 for VPC-A, 10.0.0.0/16 for VPC-B). ICMP is required for ping.



Step 5: Launch EC2 instances

Launch a t2.micro / t3.micro (free tier) instance in the public and private subnet of each VPC (4 instances in total). Use the security group created in Step 4.

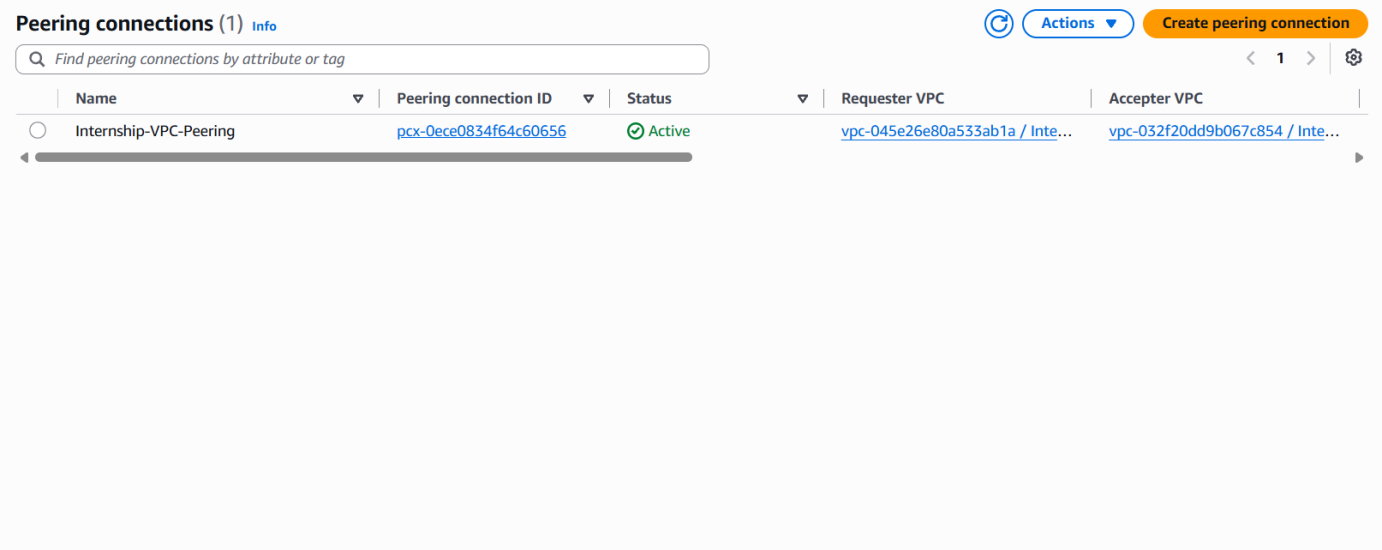


Step 6: Create the VPC Peering connection

VPC console, Peering connections, Create. Requester: VPC-A, Accepter: VPC-B. Then select the request and click **Actions, Accept request**.

Optional AWS CLI equivalent:

```
aws ec2 create-vpc-peering-connection --vpc-id <VPC-A-ID> --peer-vpc-id <VPC-B-ID>
aws ec2 accept-vpc-peering-connection --vpc-peering-connection-id <pcx-xxxx>
```

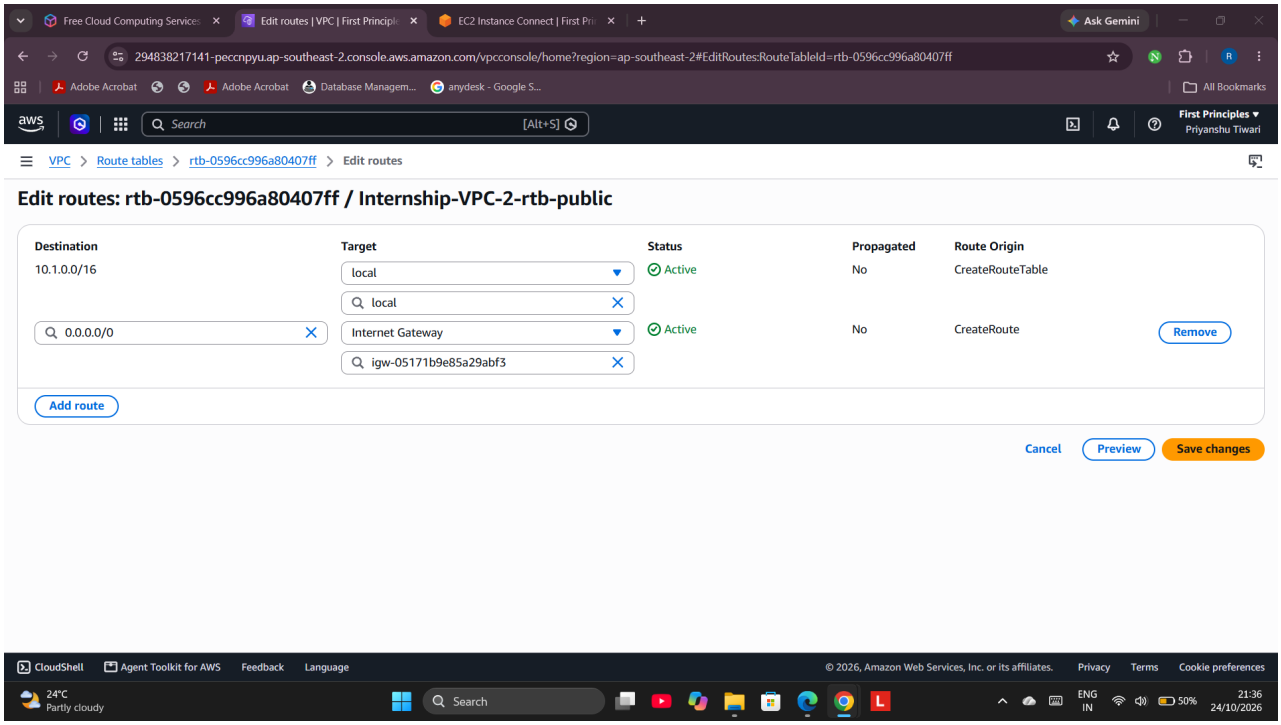


Step 7: Update route tables for peering

In VPC-A route tables add destination **10.1.0.0/16** with target the peering connection (pcx-xxxx). In VPC-B route tables add destination **10.0.0.0/16** with target the same peering connection. Do this for both public and private route tables.

Optional AWS CLI equivalent:

```
aws ec2 create-route --route-table-id <rtb-A> --destination-cidr-block 10.1.0.0/16 --vpc-peering-conne
aws ec2 create-route --route-table-id <rtb-B> --destination-cidr-block 10.0.0.0/16 --vpc-peering-conne
```

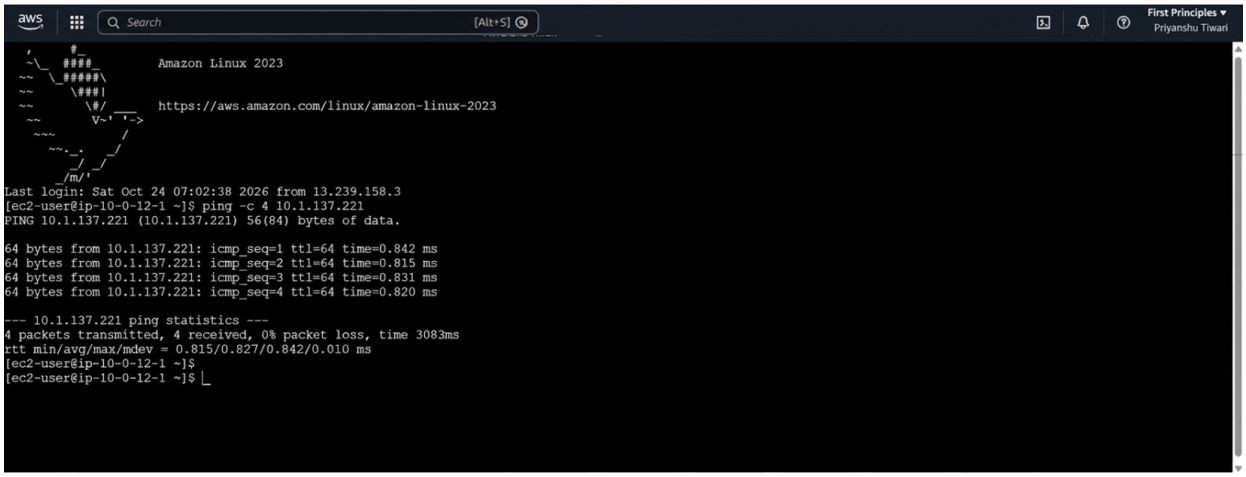


Step 8: Test connectivity

SSH into the public EC2 instance of VPC-A and ping the private IP of the EC2 instance in VPC-B. Successful replies confirm that peering works.

Optional AWS CLI equivalent:

```
ssh -i your-key.pem ec2-user@<A-public-ip>
ping <B-instance-private-ip>
```



Conclusion

Two VPCs were created with public and private subnets, connected using VPC Peering, and route tables were updated on both sides. The successful ping between EC2 instances using private IP addresses verified that the peering connection works correctly. VPC peering lets resources in different VPCs communicate privately without using the public internet.

Clean-up

After taking all screenshots, the resources were deleted in this order: EC2 instances, peering connection, internet gateways, subnets, route tables, VPCs, to avoid unwanted charges.